



Rapport NoSQL

FABRE Mathis

AMROUNI Moh-Ouali

SOUPAYA Valliama Raphael

3A IAMD

16 décembre 2025

Table des matières

1	Contexte et objectifs du projet	2
2	Architecture générale du système	2
3	Couche applicative et justification de FastAPI	2
4	Base de données orientée documents : MongoDB	3
5	Base de données orientée graphes : Neo4j	3
6	Requêtes et fonctionnalités implémentées	4
7	Annotation fonctionnelle par propagation de labels	4
8	Choix technologiques et justification	5

1 Contexte et objectifs du projet

Les données manipulées dans ce projet proviennent de Uniprot une base de donnée protéines publique qui les décrits à travers de multiples attributs : identifiant, description, taxonomie, séquence, domaines protéiques et annotations fonctionnelles (EC, GO).

Les objectifs principaux du projet sont les suivants :

- Stocker efficacement des données protéiques complexes dans une base NoSQL orientée documents.
- Construire un graphe de similarité entre protéines basé sur leurs domaines.
- Permettre des requêtes avancées sur les deux bases de données.
- Préparer le terrain pour des méthodes d'annotation fonctionnelle par propagation de labels.

Pour répondre à ces objectifs, MongoDB a été choisi comme base de données documentaire et Neo4j comme base de données orientée graphes.

2 Architecture générale du système

L'architecture du système repose sur trois composants principaux :

- Une base de données MongoDB pour le stockage des informations protéiques.
- Une base de données Neo4j pour la représentation du réseau de similarité entre protéines.
- Une couche applicative (interface graphique) chargée du traitement des données, de l'alimentation des bases et de l'exécution des requêtes.

Les données sont d'abord importées et stockées dans MongoDB sous forme de documents. À partir de ces documents, un graphe de similarité est construit puis inséré dans Neo4j. Cette séparation permet d'exploiter les points forts de chaque technologie selon les besoins.

3 Couche applicative et justification de FastAPI

La couche applicative du système repose sur le framework FastAPI, utilisé comme interface entre les bases de données MongoDB et Neo4j et les utilisateurs. FastAPI permet de concevoir rapidement des services web performants et structurés, adaptés à la manipulation de données complexes et à l'orchestration de traitements multi-bases.

L'utilisation de FastAPI présente plusieurs avantages dans le contexte de ce projet :

-
- Une définition claire et typée des requêtes grâce à l'utilisation de modèles de données, facilitant la validation des entrées utilisateur.
 - Une génération automatique d'une documentation interactive (OpenAPI / Swagger), permettant de tester facilement les différentes fonctionnalités du système sans interface graphique dédiée.
 - De bonnes performances pour des opérations de lecture et de calcul, notamment lors de la construction dynamique des sous-graphes de similarité.
 - Une intégration simple avec les bibliothèques Python utilisées pour l'accès aux bases MongoDB et Neo4j.

Dans ce projet, FastAPI joue ainsi un rôle central d'orchestrateur : il permet d'exécuter les requêtes documentaires sur MongoDB, de déclencher les calculs de similarité entre protéines et de générer dynamiquement des sous-graphes locaux dans Neo4j. Cette séparation claire entre la couche applicative et les couches de stockage améliore la lisibilité du système et facilite son évolution future.

4 Base de données orientée documents : MongoDB

MongoDB a été retenu pour sa capacité à stocker des données hétérogènes sous forme de documents JSON. Chaque protéine est représentée par un document unique regroupant l'ensemble de ses attributs biologiques.

Ce modèle permet :

- Une récupération complète des informations d'une protéine en une seule requête.
- Une grande flexibilité du schéma, adaptée à l'évolution des annotations biologiques.
- Des requêtes efficaces sur des champs imbriqués (taxonomie, domaines, annotations).

L'utilisation de MongoDB simplifie ainsi le stockage et la consultation de données biologiques complexes, sans les contraintes de normalisation imposées par les bases relationnelles.

5 Base de données orientée graphes : Neo4j

La base Neo4j est utilisée pour représenter les relations de similarité entre protéines sous forme de graphe. Chaque nœud correspond à une protéine et chaque arête relie deux protéines partageant des domaines communs.

Le poids des arêtes est calculé à l'aide de l'indice de similarité de Jaccard, défini comme

le rapport entre le nombre de domaines partagés et le nombre total de domaines distincts entre deux protéines :

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|}$$

Cette mesure permet de refléter plus fidèlement la proximité fonctionnelle entre protéines. Le graphe obtenu est non orienté et pondéré, et contient à la fois des protéines annotées et non annotées.

Dans une optique *big data*, le graphe complet des protéines peut contenir un très grand nombre de nœuds et d'arêtes, rendant toute visualisation ou analyse globale coûteuse et peu pertinente. Afin de limiter la charge computationnelle et d'améliorer l'explorabilité du graphe, notre approche repose sur la construction dynamique de *ego-graphs* à la demande.

Un ego-graph correspond à un sous-graphe centré sur une protéine d'intérêt, incluant ses voisins directs, et éventuellement les voisins de second ordre. Ce sous-graphe est extrait dynamiquement via des requêtes Cypher uniquement lors de l'interrogation d'une protéine donnée, ce qui permet de travailler sur des sous-ensembles locaux du graphe tout en évitant le chargement ou le parcours complet du graphe global.

6 Requêtes et fonctionnalités implémentées

Le système permet les fonctionnalités suivantes :

- Recherche de protéines par identifiant, nom ou description dans MongoDB.
- Visualisation des voisins pour une protéines.
- Calcul de statistiques telles que le nombre de protéines annotées, non annotées ou isolées.
- Exploration du voisinage étendu d'une protéine (voisins de voisins).

Ces fonctionnalités illustrent l'intérêt de combiner une base documentaire et une base orientée graphes dans un même système.

7 Annotation fonctionnelle par propagation de labels

L'une des motivations principales de la construction du graphe de similarité entre protéines est la possibilité de réaliser une annotation fonctionnelle automatique par propagation de labels. Cette approche repose sur l'hypothèse biologique selon laquelle des protéines partageant des domaines similaires présentent des fonctions proches.

Dans le système proposé, certaines protéines disposent déjà d'annotations fonctionnelles

(par exemple des numéros EC), tandis que d'autres sont non annotées. Le graphe de similarité permet alors d'exploiter le voisinage d'une protéine cible afin d'inférer des fonctions potentielles à partir de ses voisins les plus proches.

Bien que cette méthode d'annotation reste exploratoire dans le cadre de ce projet, elle illustre le potentiel des bases de données orientées graphes pour l'inférence de connaissances biologiques à partir de données structurales et relationnelles.

8 Choix technologiques et justification

Le choix de MongoDB et Neo4j repose sur la nature des données et les objectifs du projet. MongoDB est particulièrement adapté au stockage de documents biologiques riches et hétérogènes, tandis que Neo4j est optimisé pour l'analyse de relations complexes.

L'association de ces deux technologies permet d'obtenir un système performant, flexible et extensible, répondant aux exigences du traitement de données protéiques à grande échelle.